COURSEWORK - ST3189

Candidate number: 210530971

IRIS Dataset: Unsupervised learning 3
Primary Data Analysis: 3
K-MEANS CLUSTERING 3
DBSCAN CLUSTERING 4
AGGLOMERATIVE CLUSTERING 4
Results: 5
Dataset housing in Boston: REGRESSION 5
Random forest 5
Gradient boosting 6
Results: 6
Dataset MNIST: CLASSIFICATION 7
Logistic regression 7
Random forest 7
K-nearest neighbours 7
Results: 8

IRIS DATASET: UNSUPERVISED LEARNING

Primary Data Analysis:

The IRIS dataset contains information about three types of iris: Iris-setosa, Iris-versicolor and Iris-virginica. Each type of iris is described by four numerical signs: sepal length (cm), sepal width (cm), petal length (cm), petal width (cm). This dataset is very convenient for solving clustering problems, so I took it as a basis. An important note is that I have defined the iris species as the target, where Iris-setosa = 0, Iris-versicolor = 1 and Iris-virginica = 2.

To begin with, let's see how certain values depend on the type of iris. Figure 1 is a projection graph of each feature with a view. We see that all variables have a dependency on the species they belong to. For example, by sepal length in most cases, you can determine which type of iris belongs to.

At the same time, in order to do this, I built a heatmap (Figure 2) on which it became obvious that it is not always possible to determine by a feature what kind of iris we have, for example, sepal width has a weak correlation with all other features, especially with target. While other signs help to determine the species very well.

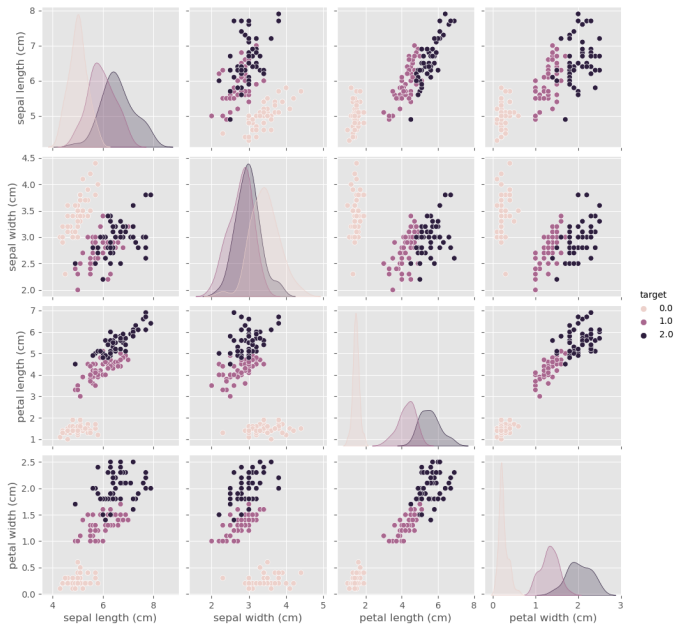

Figure 1

K-MEANS CLUSTERING

This algorithm allows to divide data into groups based on the similarity between objects. In our case, I did it based on the characteristics of iris.

The data had already been normalized, so it was not necessary to bring each feature to the same scale. The number of clusters was not difficult to determine.
Figure 2 will be three since we have only three types of irises. After running this algorithm, I realized that we can't build a graph because the dimension of our data is 4 => we will use TSNE. The main idea of this algorithm is to convert the distance between points in a high-dimensional space into the distance between points in a low-dimensional space. To do this, the algorithm uses a probabilistic model that tries to preserve the distances between points by displaying them in the probability of proximity between points in a low-dimensional space.

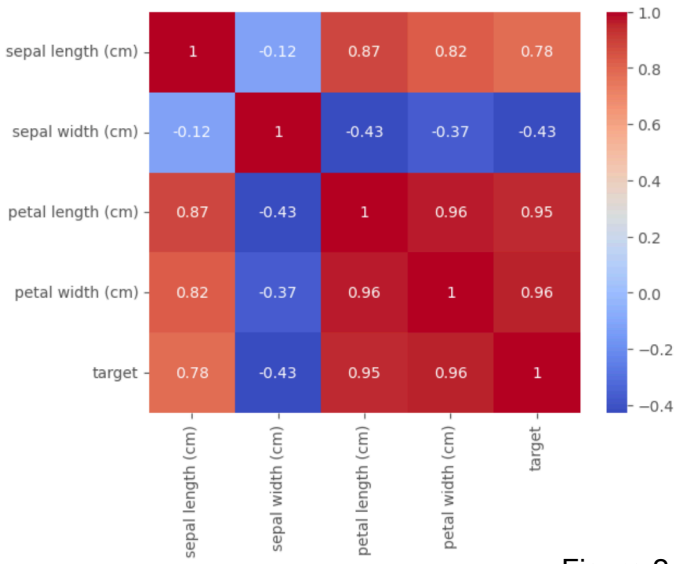

Figure 2

From the results obtained in Figure 3, we see that K-means was able to identify three clusters in the data that correspond to three types of irises: Iris-setosa, Iris-versicolor, Iris-virginica. This suggests that the data has an explicit structure and clustering can be used to isolate different types of irises. This method also helps to visualize data in order to better understand its properties and structure.

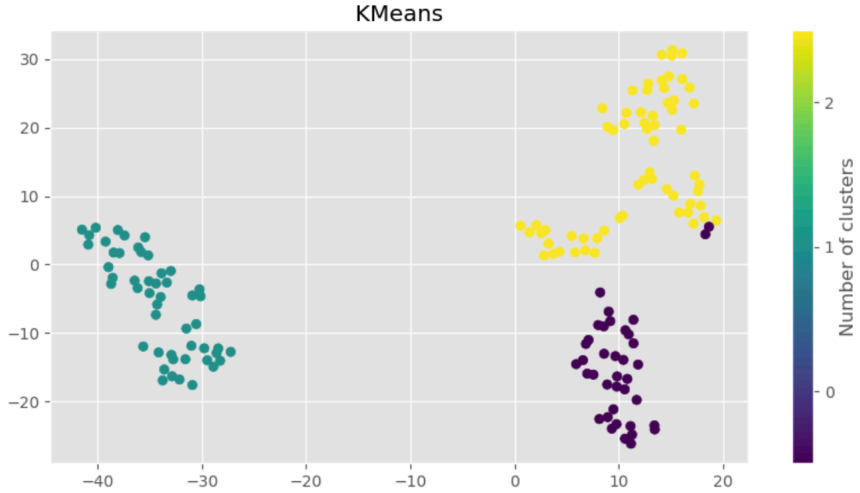

Figure 3

DBSCAN CLUSTERING

This algorithm is based on the concept of density. It allows you to automatically identify compact groups of objects and simultaneously detect outliers based on how densely the points are located in space.

In my data, it also clusters based on features such as the length and width of the petals and sepals. The advantages of this algorithm is that it identifies clusters of arbitrary shape, unlike K-means, which identifies only convex clusters. But as we see in Figure 4, we see that this method also has disadvantages, so it requires more fine tuning. Since this algorithm depends on parameters such as the radius and the minimum number of points, which just affect the results of clustering.

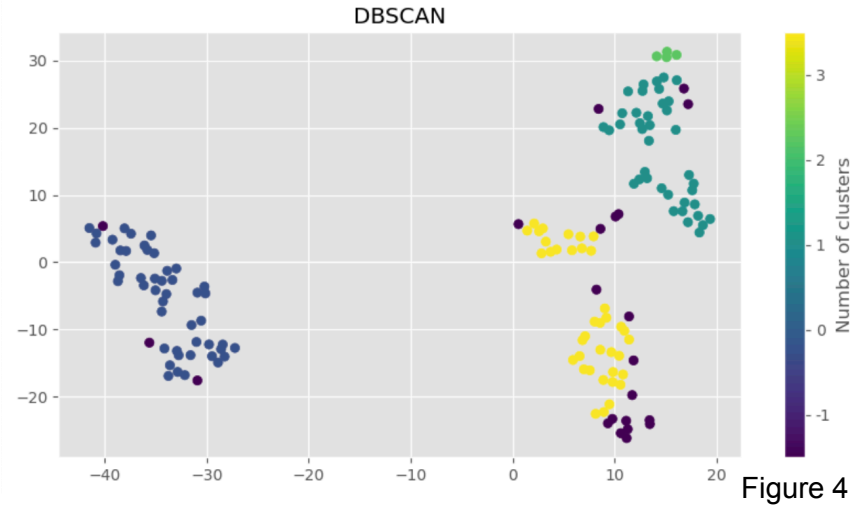

Figure 4

AGGLOMERATIVE CLUSTERING

This is a hierarchical clustering method that starts with each characteristic of each object in a separate class and combines them into larger clusters. Why I chose this method, it has advantages since it is not necessary to specify the number of clusters, unlike K-means and DBSCAN, where the parameters must be defined in advance.Also, this algorithm is resistant to noise and emissions, it can take them into account if necessary. The result obtained in Figure 5 shows that clusters are also easily determined by the agglomerative clustering algorithm.

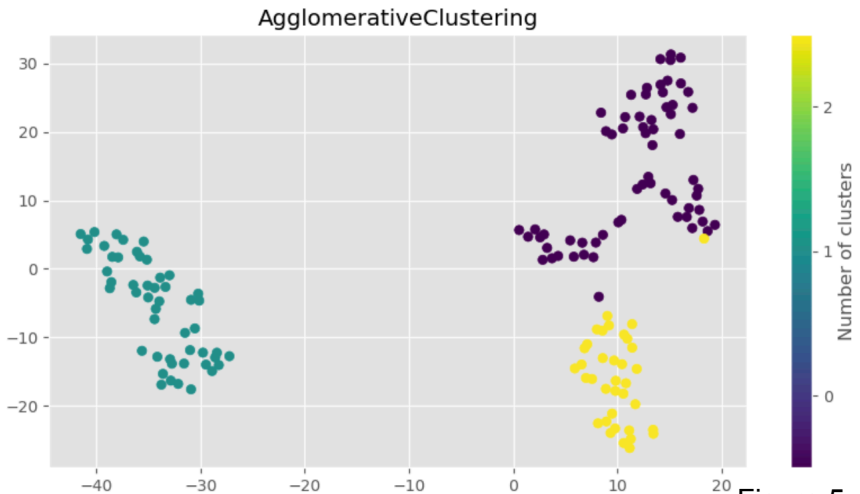

Figure 5

Results:

If K-means and Agglomerative Clustering have successfully allocated clusters in the Iris database, and DBSCAN requires configuration, then we can conclude that the data structure is most likely suitable for these methods.

This may be due to the fact that DBSCAN may be more sensitive to parameters such as radius and minimum number of points that define clusters. At that time, K-means and Agglomerative Clustering are simpler methods that can work well on different types of data without tweaking the parameters. It so happened that these methods successfully identified clusters in the Iris dataset, which indicates a good division of data into clusters and their similarity within them.

DATASET HOUSING IN BOSTON: REGRESSION

Random forest

To do this, I needed to use other data, namely, I decided to take a data set on housing in Boston, which is compiled on the basis of information collected by the US Census Service on housing in the Boston, Massachusetts area. As a target variable, I chose a price expressed in hundreds of thousands of dollars ($100,000). To begin with, I compiled a correlation matrix for the assessment (Figure 6), in which it became clear the relationship between target and other variables, so the most important point affecting the cost of housing was the average income. So in order to predict housing prices, I decided to use the random forest algorithm, which is a combination of many decision trees. Each tree makes a decision based on a set of random features and their threshold values.

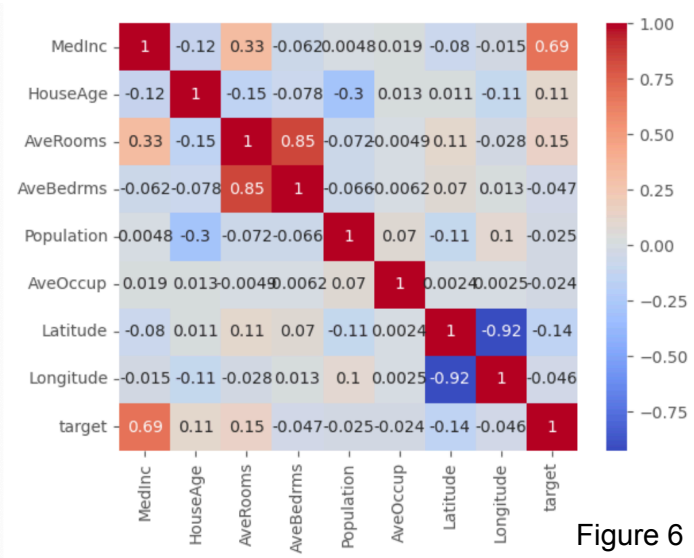

Figure 6

To begin with, it was necessary to determine the optimal number of trees, so I used the validation curve method. This method has shown that the optimal number of trees is 100, which we can observe in Figure 7.

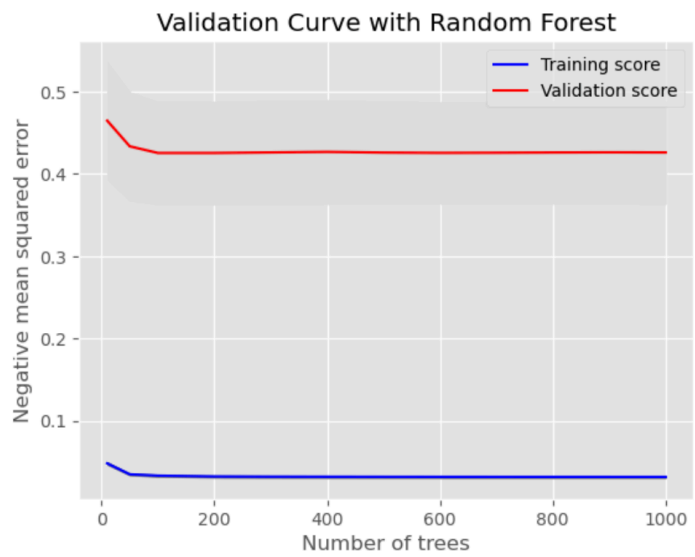

Figure 7

```
Optimal number of estimators: 100
```

So I applied the RMSE method, which is very convenient for checking the quality of models, I got 0.5089. In general, RMSE estimates the standard deviation between the model predictions and the actual values of the target variable, then this small value indicates that the model predicts the target variable well on the test sample. Next I applied R^2 and got the result 0.75. This made it clear that the features used in the model explain the variation in the target variable well.

Next, I decided to look at the dependence of the error on the training and test sets on the number of trees and got Figure 8. And a problem arose since the error on test is greater than

on train, so there is a need of slight retraining. In order not to change the entire model, I decided to use the "warm_start" parameter. After retraining, I received Figure 9. Where it is clear that retraining does not grow linearly.

Figure 8

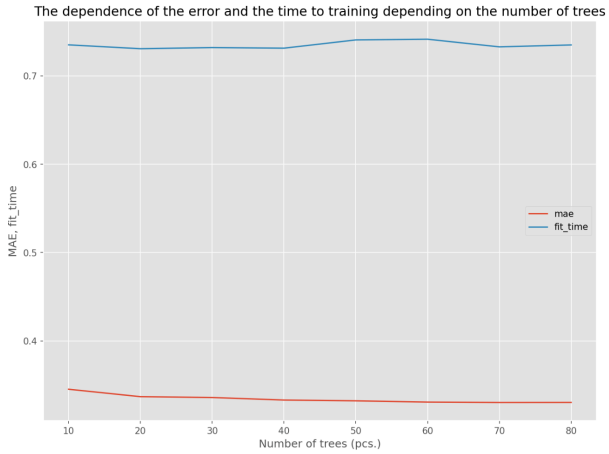

Figure 9

Further, for a better understanding, I decided to deduce how my model makes a decision depending on the data received, so I built a tree of depth 3 in order to check the correctness of the algorithm. As a result, I got that my model successfully makes a decision (Figure 10). It also became obvious that now it is possible to use this model for classification, which I actually did with gradient boosting.

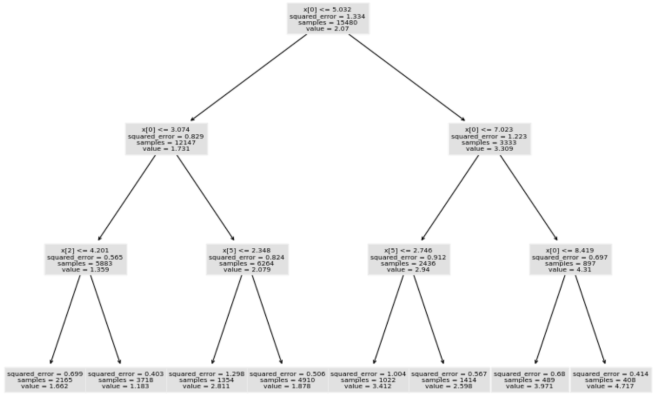

Figure 10

Gradient boosting

Next, let's solve the same problem using gradient boosting. This method is used to solve classification and regression problems. It is based on the idea of combining several simple machine learning models into one more complex one. This method allows you to achieve high accuracy of predictions, including those cases when individual models cannot cope with the task. After applying gradient boosting, I got that the coefficient of determination (R^2) is 0.82. This means that the model accurately explains the change in the dependent variable (housing prices) depending on the data set. This value says that gradient boosting is able to explain 82% of the data variance. However, I decided to make an additional analysis and calculate the RMSE and got a value equal to 1.42, which means that the average error of the model's predictions is 1.42 thousand dollars. In general, the lower the RMSE value, the better the quality of the model. However, this value can be considered as a pretty good result, especially considering that there are housing prices in the dataset that differ significantly from the average.

Results:

Thus, it can be concluded that both models have a fairly good prediction quality, and gradient boosting has shown itself to be better than a random forest. Also, based on the RMSE value, it can be concluded that in the case of a random forest, the model predicts housing prices fairly accurately, and the average error is only 0.5089 thousand dollars. However, in the case of gradient boosting, the average prediction error of the model is higher and amounts to 1.42 thousand dollars. That is, if the most important criterion is the accuracy of predictions, then it is better to use a

random forest. However, if the overall quality of the model is important, it is better to use gradient boosting, since it provides a higher value of the determination coefficient.

DATASET MNIST: CLASSIFICATION

Logistic regression

Now let's move on to the classification problem, for this I decided to use the MNIST dataset. It consists of a data set containing handwritten numbers from 0 to 9. I decided that I could try to apply logistic regression, because this method is used to predict the probability of a certain event based on the given signs. Without normalization, we get that the accuracy is 0.89, which means that our model recognizes numbers quite well. After that, I applied normalization, namely normalized the pixel values of the input images. This helped to improve the training and accuracy of the model to 0.93.

Random forest

To begin with, I decided to think over the number of etimators. To do this, I used cross-validation, which allows you to evaluate the quality of the model on several data differences and average the resulting metrics. I split this dataset into its attributes and the target variable, then ran a random forest and determined the range from 10 to 100. As a result, I got that the value of 50 is the maximum optimal value and deduced that with these parameters, the accuracy of predicting the figure is 0.9399. I also built a graph on which I described how the accuracy changes depending on the number of estimator (Figure 11)

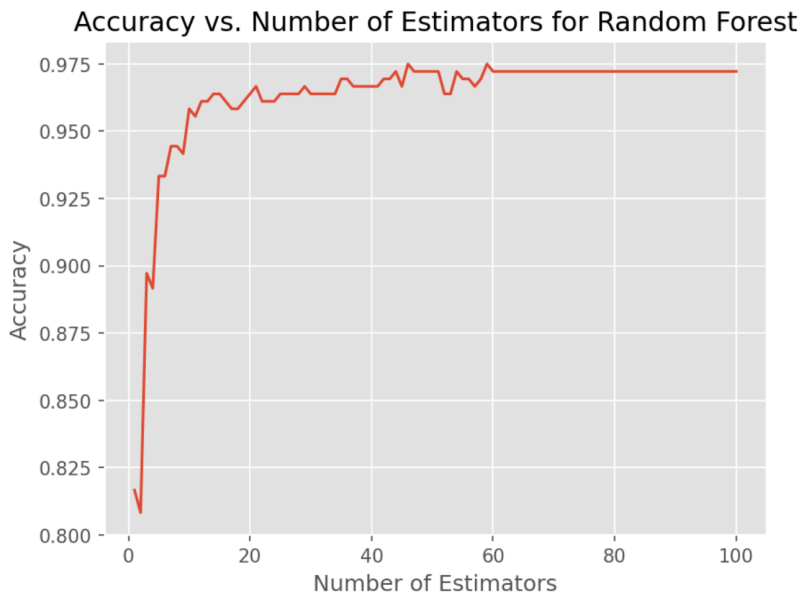

Figure 11

K-nearest neighbours

I also suggested that this particular model might be better for this dataset. Therefore, I decided to use predictions on a test sample for each value of neighbours starting from 1 and ending with 10. The result can be seen in Figure 12.

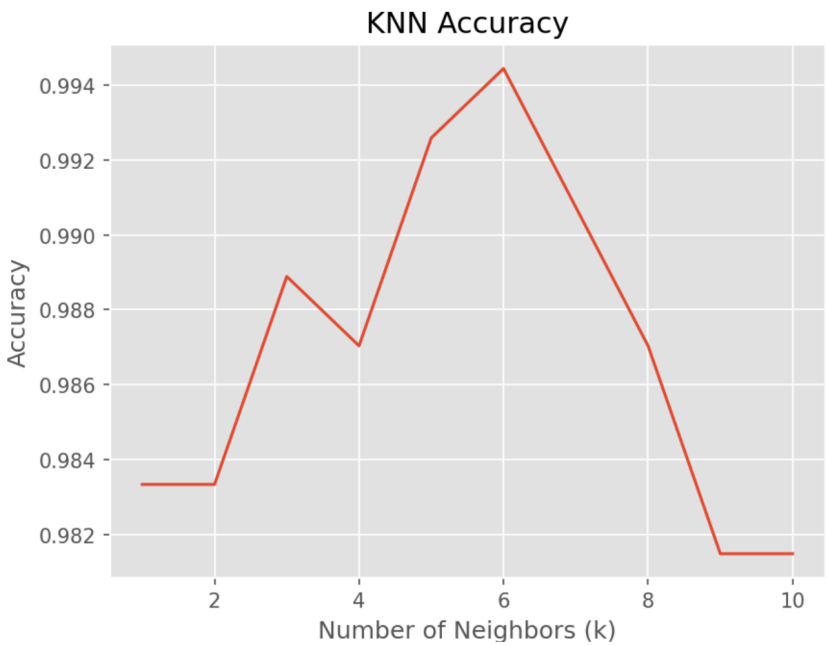

Figure 12

Results:

Surprisingly, the KNN model turned out to be the most accurate, apparently it is most suitable for prediction on this data set, despite this, it must be borne in mind that much depends on how the model is configured and whether it is checked for correctness. At the same time, KNN requires a large amount of memory and time to perform predictions on large datasets. In Figure 13, we see how the logistic regression, random forest, and KNN models are accurate.

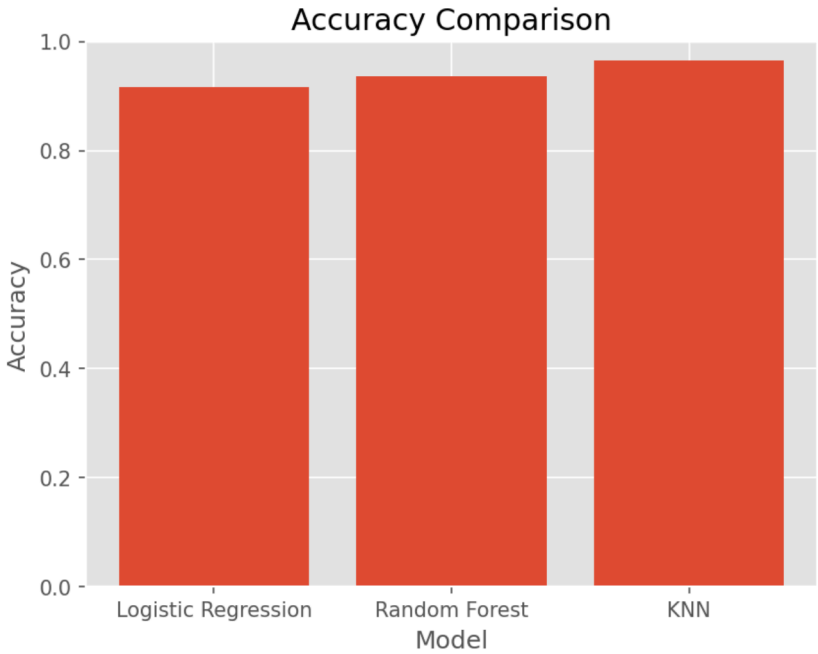

Figure 13